

Documentación del proyecto

Arquitectura, datos, seguridad, API, instalación, pruebas y despliegue.

Angular

TypeScript

MySQL

Gestión circular y trazabilidad RAEE

Publicar hardware, reservarlo y confirmar su entrega para registrar su recuperación e impacto ambiental.

REFERENCIA TÉCNICA

Guía de lectura y alcance

Documento para desarrolladores, responsables de instalación y evaluadores. Describe el estado local revisado el 9 de octubre de 2026, sin modificar la aplicación ni ejecutar cambios sobre la base de datos.

Objetivos y requisitos	3
Arquitectura y monorrepositorio	4
Modelo de datos	5
Autenticación y autorización	6
Proceso transaccional	7
Frontend y experiencia	8
API: Auth y dispositivos	9
API: transferencias y métricas	10
API: categorías y centros	11
Instalación de desarrollo	12
Entorno, migración y seed	13
Pruebas y validación	14
Producción y operación	15
Límites y mantenimiento	16

Fuentes

README.md, package.json, backend/src, frontend/src/app, DB/migrations, pruebas y runners. No se leen ni publican los valores del .env real.

Validación registrada

El README actual registra 72 pruebas unitarias y 14 casos de integración. Este documento atribuye esos resultados al registro previo; no los presenta como una ejecución nueva.

01 · CONTEXTO

Objetivos y requisitos

EcoPlaca gestiona hardware en desuso para facilitar recuperación, reutilización y trazabilidad. Su vínculo con ODS 11 y 12 se basa en la gestión de residuos y el uso responsable de recursos.

Requisito trabajado	Evidencia
RF 08 · CRUD	Dispositivos, categorías y centros; validación M:N y baja lógica.
RF 09 · Proceso	Reserva y recepción en transacciones; Catálogo y Mis transferencias.
RF 10 · Filtros	Debounce y filtros de estado/condición; SQL parametrizado.
RF 11 · Métricas	Kg, CO ₂ estimado y transferencias por estado.
RF 12 / RF 15 · Feedback	Login, registro, validaciones, carga y errores visibles.
RF 14 · Confirmación	Diálogos de reserva y recepción irreversible desde el panel.
RF 16 · Sesión	Logout, expiración, restauración y cambios entre pestañas.

Proceso verificable

01 Donante Publica	02 Técnico Reserva	03 Recepción Confirma	04 Dashboard Mide
--	--	---	---------------------------------------

La publicación toma al donante del JWT, exige un centro compatible y asigna un código de trazabilidad único. La reserva aparta el dispositivo; la entrega o el estado Reciclado permiten incorporar peso al total recuperado.

Relación con la guía

Esta tabla relaciona requisitos mencionados en las tareas con componentes concretos. No sustituye una auditoría completa de una guía externa de evaluación.

02 · DISEÑO

Arquitectura y monorrepositorio

Angular

Componentes · Signals · Forms
Guards · Interceptores · Servicios HTTP

HTTP / JSON + Authorization: Bearer

Express / TypeScript

Rutas · JWT · Roles · Validación
Controladores · Transacciones

Pool mysql2 / SQL parametrizado

MySQL · InnoDB

7 tablas · Claves foráneas · M:N
Estados, peso y trazabilidad

Ubicación	Responsabilidad
backend/src/config	Entorno, CORS y pool MySQL.
controllers	Endpoints, reglas, publicación, órdenes, métricas y CRUD.
middleware	JWT, roles y errores.
models / routes / utils	Contratos, enrutamiento, validación y CO ₂ .
backend/src/scripts	Migración, seed y bloqueo de preparación.
frontend/src/app	Componentes, modelos, servicios, Signals, guards e interceptores.
DB/migrations	Esquema y actualización de estados.
scripts / backend/scripts	Runners npm de pruebas.

Stack: Angular 19.2, RxJS 7.8, TypeScript 5.7, Express 4.21, mysql2, jsonwebtoken y bcryptjs. El README toma como referencia Node.js 22.14.0, npm 10.9.2 y MySQL 8.0.

03 · MYSQL / INNODB

Modelo de datos

Tabla	Clave y función
roles	PK id; nombre único. ADMIN, DONOR y TECHNICIAN.
usuarios	PK id; FK rol_id; email único, password_hash y activo.
categorias_raee	PK id; código/nombre únicos; factor_co2_kg.
centros_acopio	PK id; ubicación, capacidad_kg y activo.
centros_categorias	PK centro_id + categoria_id; dos FKs.
dispositivos	PK id; FKs categoría/donante/centro; código único, estados, peso y CO ₂ .
ordenes_transferencia	PK id; FKs dispositivo/técnico/origen/destino; estado, motivo y fechas.

Relaciones

Origen	Relación	Destino
roles	1:N	usuarios
usuarios (donante)	1:N	dispositivos
categorias_raee	1:N	dispositivos
centros_acopio	1:N	dispositivos
centros_acopio	M:N mediante centros_categorias	categorias_raee
dispositivos	1:N histórico	ordenes_transferencia
usuarios (técnico)	1:N	ordenes_transferencia

El diseño en 3FN separa los atributos de roles y categorías de usuarios/dispositivos. La tabla puente almacena pertenencia centro/categoría. Las restricciones referenciales protegen los vínculos.

Conservación

Una categoría vinculada no se elimina. Un centro se desactiva. Un dispositivo solo se edita/elimina si está Disponible, no tiene órdenes y pertenece al donante o interviene ADMIN.

04 · SEGURIDAD

Autenticación y autorización

01

Credenciales

Login

02

BCrypt

Hash

03

JWT

Token

04

Middleware

Firma + rol

Contrato

- Registro acepta rolId 2/3; no permite ADMIN público.
- Login compara BCrypt y devuelve data.token y data.usuario.
- Perfil recupera datos activos; Angular normaliza sus campos.
- JWT_EXPIRES_IN acepta segundos o s/m/h/d/w; ejemplo 7d.
- authorizeRoles consulta el rol y actividad actuales en MySQL.

Sesión y guards

AuthService expone currentUser e isAuthenticated con Signals, guarda JWT en localStorage, restaura con el perfil y limpia estado en logout/expiración. Los eventos de almacenamiento sincronizan cambios entre pestañas.

authGuard redirige a /login con returnUrl. roleGuard aplica data.roles y vuelve al Catálogo con aviso si faltan permisos. El interceptor añade JWT solo a la API configurada, excluyendo login y registro.

Límite de localStorage

Es accesible a JavaScript y no ofrece protección HttpOnly. Los guards no sustituyen la autorización del servidor. No hay refresh tokens ni recuperación de contraseña en pantalla.

Entorno

DB_HOST, DB_USER, DB_PASSWORD y DB_NAME son obligatorios. JWT_SECRET es obligatorio y requiere al menos 32 caracteres en producción. CORS acepta orígenes explícitos; no autoriza operaciones por rol.

05 · RESERVA / ENTREGA

Proceso transaccional

Solicitar transferencia

1. Validar dispositivo, motivo y destino opcional; obtener conexión.
2. Iniciar beginTransaction y bloquear dispositivo con FOR UPDATE.
3. Exigir DISPONIBLE, ausencia de orden activa y centro de origen.
4. Validar centros activos y compatibles con la categoría.
5. Crear orden PENDIENTE y cambiar dispositivo a RESERVADO.
6. Commit; rollback ante error; liberar conexión en finally.

Completar transferencia

1. Validar ID y localizar dispositivo de la orden.
2. Bloquear primero dispositivo y luego orden para conservar el orden de bloqueo.
3. Exigir técnico responsable o ADMIN, orden PENDIENTE/EN_TRANSITO y dispositivo RESERVADO.
4. Cambiar a COMPLETADA con fecha_completado y dispositivo ENTREGADO.
5. Aplicar centro destino cuando corresponde y hacer commit.
6. Repetición o estado incompatible: 409 sin duplicar el impacto.

Concurrencia

Los bloqueos impiden dos reservas simultáneas exitosas del mismo hardware. Los errores concurrentes revierten la transacción y permiten informar el conflicto.

Métricas

La consulta agrega peso y CO₂ de dispositivos recuperados una vez, sin multiplicarlos por órdenes históricas. Los conteos de transferencias por estado se obtienen mediante la misma sentencia SQL.

06 · ANGULAR

Frontend y experiencia

Ruta	Acceso	Comportamiento
/catalogo	Público	Filtros, carga, vacío, errores y reservas.
/login, /register	Público	Formularios y feedback HTTP.
/nuevo-dispositivo	DONOR, ADMIN	Publicación con opciones de MySQL.
/transferencias	TECHNICIAN, ADMIN	Órdenes y recepción física.
/dashboard	Los tres roles	Métricas manuales y cada 30 s.
/comentarios	TECHNICIAN, ADMIN	Notas locales no persistidas.
/ y /galeria	Redirección	Destino /catalogo.
Ruta desconocida	Público	404 y regreso al Catálogo.

Reactividad

Signals guardan sesión, resultados, selección y estados visuales. Reactive Forms valida formularios. El catálogo combina filtros, aplica debounce de 350 ms y switchMap cancela consultas anteriores.

Mutaciones

TransferenciasService emite cambios\$ después del éxito. La vista actualiza COMPLETADA/ENTREGADO y retira la orden activa. El Dashboard consulta al abrirse y reacciona al evento si está montado; otros clientes se reflejan en la próxima consulta.

Accesibilidad y diseño

Paleta #1B4965 y #2EC4B6. Navegación responsiva y por teclado, formularios etiquetados, errores visuales y diálogos nativos que protegen envíos en curso.

Persistencia

Las notas del dispositivo pertenecen a MySQL. La pantalla Comentarios y notas usa demostración/estado local y avisa que lo añadido se pierde al salir o recargar.

07 · ENDPOINTS · PARTE 1

API: Auth y dispositivos

Base local: `http://localhost:3000/api`. JSON en Content-Type; para rutas protegidas, Authorization: Bearer <token>.

Método / ruta	Acceso	Función
GET /health	Público	Salud de la API.
POST /auth/register	Público	Crear DONOR/TECHNICIAN.
POST /auth/login	Público	JWT y usuario.
GET /auth/perfil	JWT	Perfil activo.
GET /dispositivos	Público	Listar y filtrar.
GET /dispositivos/:id	Público	Detalle.
GET /dispositivos/opciones-publicacion	DONOR / ADMIN	Categorías y centros.
POST /dispositivos	DONOR / ADMIN	Publicar con donante del JWT.
PUT /dispositivos/:id	Propietario / ADMIN	Editar disponible sin historial.
DELETE /dispositivos/:id	Propietario / ADMIN	Retirar sin historial.
PATCH /dispositivos/:id/estado	ADMIN	Estado y notas.

Publicación — ejemplo

```
{
  "titulo": "Placa madre para reparación",
  "categoriaId": 1,
  "centroAcopioId": 1,
  "estadoFuncional": "REPARABLE",
  "pesoKg": 1.25,
  "notas": "Requiere diagnóstico"
}
```

Usar IDs reales de opciones-publicacion. El listado admite búsqueda, estado, estadoFuncional y categoriaId. El centro de publicación debe estar activo y ser compatible.

08 · ENDPOINTS · PARTE 2

API: transferencias y métricas

Método / ruta	Acceso	Función
POST /transferencias	TECHNICIAN	Reserva.
GET /transferencias/mis-ordenes	TECHNICIAN / ADMIN	Propias o todas.
PATCH /transferencias/:id/completar	Responsable / ADMIN	Recepción.
GET /dashboard/metricas	Los tres roles	Peso, CO ₂ y estados.

Solicitud — ejemplo

```
{
  "dispositivoId": 123,
  "motivo": "Reacondicionamiento para reutilización",
  "centroDestinoId": 2
}
```

centroDestinoId es opcional y, si se envía, debe ser compatible. mis-ordenes no deja seleccionar las órdenes de otro técnico mediante parámetros.

Métricas — ejemplo ilustrativo

```
{
  "success": true,
  "data": {
    "totalKgRecuperados": 1.25,
    "co2EvitadoKg": 44.38,
    "transferenciasPorEstado": {
      "PENDIENTE": 1,
      "EN_TRANSITO": 0,
      "RECIBIDO": 0,
      "CANCELADO": 0,
      "COMPLETADA": 1
    }
  }
}
```

Se suma peso_kg y co2_evitado_kg de estados ENTREGADO/RECICLADO. El cálculo de cada dispositivo usa la utilidad ambiental y el factor de categoría.

Conteos

La API conserva RECIBIDO/CANCELADO históricos. Angular suma PENDIENTE + EN_TRANSITO + COMPLETADA en su tarjeta de transferencias.

Errores habituales

400: datos; 401: sesión; 403: rol/propiedad; 404: entidad; 409: estado, vínculos, duplicación o concurrencia.

09 · ENDPOINTS · PARTE 3

API: categorías y centros

Método / ruta	Acceso	Función
GET /categorias, /categorias/:id	Público	Listar / detalle.
POST /categorias	ADMIN	Crear.
PUT /categorias/:id	ADMIN	Actualizar e impacto relacionado.
DELETE /categorias/:id	ADMIN	Eliminar sin vínculos.
GET /centros, /centros/:id	Público	Listar / detalle y categorías.
POST /centros	ADMIN	Crear y relacionar.
PUT /centros/:id	ADMIN	Datos y categorías admitidas.
PATCH /centros/:id/desactivar	ADMIN	activo = false.

Contratos — ejemplos

```
{
  "codigo": "RAEE-TEST",
  "nombre": "Categoría de prueba",
  "descripcion": "Opcional",
  "factorCo2Kg": 30.5
}
```

```
{
  "nombre": "Centro de prueba",
  "direccion": "Dirección real",
  "ciudad": "Guatemala",
  "telefono": null,
  "capacidadKg": 500,
  "categoriasIds": [
    1,
    2
  ]
}
```

PUT actualiza campos editables completos. Los IDs deben existir y ser positivos/únicos. Las relaciones M:N cambian en transacción; no pueden retirarse categorías de dispositivos disponibles/reservados del centro.

Acceso por API

Los CRUD administrativos no tienen una pantalla Angular. Desactivar conserva el historial y las categorías vinculadas no se eliminan.

10 · PUESTA EN MARCHA

Instalación de desarrollo

Requisito	Referencia
Node.js / npm	22.x / 10.x; registro 22.14.0 / 10.9.2.
MySQL	8.0, base creada y permisos para migrar.
Chrome / Edge	Pruebas frontend headless.

Repositorio y dependencias

```
git clone https://github.com/Pablo-Rf8/EcoPlaca.git
cd EcoPlaca
npm ci
```

Base y entorno

```
CREATE DATABASE IF NOT EXISTS ecoplaca_db
CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
```

Ejecutar en MySQL con una cuenta autorizada. Asignar un usuario a la base. Copiar backend/.env.example a backend/.env y completar datos propios sin sobrescribir un archivo existente.

```
PORT=3000
NODE_ENV=development
DB_HOST=localhost
DB_PORT=3306
DB_USER=USUARIO_LOCAL
DB_PASSWORD=PASSWORD_LOCAL
DB_NAME=ecoplaca_db
JWT_SECRET=CLAVE_ALEATORIA_PRIVADA
CORS_ORIGINS=http://localhost:4200
```

Preparar y arrancar

```
cd backend
npm run build
npm run db:prepare
npm run dev
```

En otra terminal, desde frontend/:

```
npm start
```

Frontend: <http://localhost:4200>. API: <http://localhost:3000/api>; salud </api/health>. Ajustar DB_PORT si MySQL usa 3308. El proxy de desarrollo dirige </api> a 3000.

11 · CONFIGURACIÓN

Entorno, migración y seed

Variable	Uso
DB_HOST / DB_PORT	Servidor MySQL y puerto.
DB_USER / PASSWORD / NAME	Credenciales privadas y base obligatorias.
JWT_SECRET	Firma; ≥32 caracteres en producción.
JWT_EXPIRES_IN	Segundos o sufijo s/m/h/d/w; ejemplo 7d.
CORS_ORIGINS	Orígenes exactos, separados por comas; HTTPS en producción.
SEED_ADMIN_EMAIL / PASSWORD	ADMIN opcional; contraseña ≥12 caracteres y ≤72 bytes.
SEED_ADMIN_NOMBRE	Nombre del administrador.
SEED_CENTRO_NOMBRE / DIRECCION / CIUDAD	Completar los tres para crear centro inicial real.

Migración

db:migrate ejecuta dist/scripts/migrate.js. 001 crea las siete tablas con CREATE TABLE IF NOT EXISTS. 002 añade ENTREGADO/COMPLETADA si faltan en los ENUM. Se verifica information_schema y se usa GET_LOCK para evitar preparación simultánea.

Seed

Crea roles y categorías faltantes; ADMIN opcional solo si no existe. No promueve cuentas ni restablece contraseñas. DONOR/TECHNICIAN se registran por API. El centro inicial existente conserva su configuración.

Protección de datos

DB/ecoplaca_DB.sql contiene DROP DATABASE. No usarlo con datos que deban conservarse. El despliegue usa DB/migrations, no reinicializa la base.

DDL

MySQL puede hacer commit implícito en cambios de esquema. Las migraciones verifican estado y son repetibles; no prometen rollback total del DDL. El seed sí agrupa cambios de datos en transacción.

12 · CALIDAD

Pruebas y validación registrada

El README del 9 de octubre de 2026 registra los siguientes resultados. Durante esta elaboración no se volvieron a ejecutar las suites.

Registro	Resultado / alcance
Unitarias backend	36 aprobadas; MySQL simulado.
Unitarias Angular	36 aprobadas; HTTP simulado y headless.
Integración real	14 casos; 15 pruebas contando el grupo del runner. MySQL local 8.0.34.
Revisión frontend	33 escenarios a 320, 768 y 1440 px, con datos simulados.
Build y recorrido	Compilación y recorrido de publicación, reserva, entrega y métricas registrados.

Comandos desde raíz

```
npm test
npm run test:backend
npm run test:frontend
npm run build
```

Los runners unitarios configuran variables y no requieren una base real ni .env. El frontend usa ChromeHeadless; en Windows puede detectar Edge. Configurar CHROME_BIN si no encuentra navegador.

Integración

```
npm run test:integration
```

Preparar antes API local con base exclusiva ecoplaca_qa o un nombre terminado en _test, migración/seed y ADMIN de prueba. API_URL admite localhost; por defecto http://127.0.0.1:3000.

Datos de prueba

La integración crea datos sintéticos únicos y los conserva. No reinicializa la base. Ejecutar contra la base QA autorizada, no sobre producción.

Referencias: backend/tests/*.test.ts, backend/scripts/test.cjs, scripts/test-frontend.cjs y backend/api.http.

13 · DESPLIEGUE

Producción y operación

1. Configurar y compilar

Crear base y credenciales. Definir `NODE_ENV=production`, `JWT_SECRET`, `JWT_EXPIRES_IN` y `CORS_ORIGINS` con dominio HTTPS real. Instalar dependencias de desarrollo durante la construcción.

```
npm ci
npm run build
```

2. Preparar base y arrancar

Desde backend/:

```
npm run deploy
```

Encadena `db:prepare` y `npm start`. Una preparación fallida impide arrancar. Puede ejecutarse `db:prepare` como etapa separada y `npm start` como proceso de servicio.

3. Publicar Angular

- Servir `frontend/dist/frontend/browser` con HTTPS.
- Reescribir rutas Angular a `index.html` para acceso directo.
- Enviar `/api` al backend conservando el prefijo y antes del fallback Angular.
- Conservar `backend/dist`, dependencias de producción y `DB/migrations` con posición relativa del repositorio.

4. API en otro origen

Proporcionar `API_URL` en `frontend/src/app/app.config.ts`, recompilar y permitir el origen del frontend en `CORS_ORIGINS`. `proxy.conf.json` solo configura desarrollo.

5. Operación

Comprobar salud, login y el ciclo completo con cuentas autorizadas. El servidor valida entorno y conectividad antes de escuchar y maneja `SIGTERM/SIGINT` para cerrar HTTP/pool. Añadir respaldo y monitoreo según la instalación.

Secretos

No usar contraseñas demo en producción. Este documento no incluye ni modifica el `.env` real.

14 · REFERENCIAS

Límites y mantenimiento

Alcance actual

- Sin WebSockets/SSE: cambios de otros clientes llegan en la próxima consulta.
- CRUD de categorías/centros y edición/retiro de hardware por API, sin pantallas.
- Sin recuperación de contraseña, refresh tokens, chat ni agenda.
- Comentarios y notas de taller es local; no persiste en MySQL.
- CO₂ es una estimación configurable, no una certificación.

Mantenimiento

1. Revisar contratos y formatos antes de añadir campos o endpoints.
2. Mantener SQL parametrizado, validación y autorización por rol/propiedad.
3. Proteger historial y probar concurrencia y conflictos.
4. Actualizar README, entorno de ejemplo y documentación si cambia el comportamiento.
5. Ejecutar pruebas y validar en QA antes de producción.

Fuentes

Referencia	Ubicación
Arranque y validaciones registradas	README.md
Contratos / implementación	backend/src y frontend/src/app
Esquema	DB/migrations/001_schema.sql y 002_transferencias_dashboard.sql
Recorrido manual	backend/api.http
ODS 11 / 12	sdgs.un.org/goals/goal11 y sdgs.un.org/goals/goal12
Compatibilidad Angular	angular.dev/reference/versions

Edición

Versión documental 1.0 · 9 de octubre de 2026. Basada en código local. Los resultados previos se atribuyen al README; la generación del PDF no certifica un despliegue nuevo.